Basic Network Sniffer Project Report

Prepared by: E Arvind Udiyar
Course: MSc Computer Science

---

1. Introduction

A Network Sniffer is a cybersecurity tool used to capture and analyze data packets flowing across a network interface. It helps in understanding how communication occurs between devices and how protocols such as TCP, UDP, and ICMP operate. In this project, Python and the Scapy library were used to capture live network traffic and display important packet details including source IP, destination IP, protocol number, port numbers, and payload data.

2. Working of the Program

The program utilizes the sniff() function from Scapy to monitor network traffic. Each captured packet is processed through a callback function that checks whether the packet contains an IP layer. If TCP, UDP, or ICMP layers are detected, their respective port numbers and protocol types are extracted and displayed. Administrator privileges are required because packet capturing involves low-level access to the network interface card (NIC).

3. Output Analysis

Based on the execution screenshots, local IP addresses such as 192.168.x.x indicate private network communication. Protocol number 17 corresponds to UDP traffic, while protocol number 6 corresponds to TCP traffic. Port 443 represents secure HTTPS communication between the local system and external servers. This demonstrates real-time packet transmission occurring within the network environment.

4. Program Execution Screenshots

```
C:\Windows\system32\cmd.e:

                              (a)bort.
  --trusted-host <hostname>   Mark this host or host:port pair as trusted, even though it does not have valid or any
                              HTTPS.
  --cert <path>               Path to PEM-encoded CA certificate bundle. If provided, overrides the default. See 'SSL
                              Certificate Verification' in pip documentation for more information.
  --client-cert <path>        Path to SSL client certificate, a single file containing the private key and the
                              certificate in PEM format.
  --cache-dir <dir>           Store the cache data in <dir>.
  --no-cache-dir              Disable the cache.
  --disable-pip-version-check
                              Don't periodically check PyPI to determine whether a new version of pip is available for
                              download. Implied with --no-index.
  --no-color                  Suppress colored output.
  --use-feature <feature>     Enable new functionality, that may be backward incompatible.
  --use-deprecated <feature>  Enable deprecated functionality, that will be removed in the future.
  --resume-retries <resume_retries>
                              Maximum attempts to resume or restart an incomplete download. (default: 5)

C:\Users\ADMIN>pip install scapy
Collecting scapy
  Downloading scapy-2.7.0-py3-none-any.whl.metadata (5.8 kB)
Downloading scapy-2.7.0-py3-none-any.whl (2.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 1.6 MB/s  0:00:01
Installing collected packages: scapy
Successfully installed scapy-2.7.0

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\ADMIN>
```

Py.py - Z:\Arvind\Py.py (3.13.7)

File Edit Format Run Options Window Help

```python
from scapy.all import sniff, IP, TCP, UDP, ICMP

def packet_callback(packet):

    if packet.haslayer(IP):
        ip_layer = packet[IP]
        print("\n--- New Packet Captured ---")
        print("Source IP:", ip_layer.src)
        print("Destination IP:", ip_layer.dst)
        print("Protocol:", ip_layer.proto)

        # Check Protocol Type
        if packet.haslayer(TCP):
            print("Protocol Type: TCP")
            print("Source Port:", packet[TCP].sport)
            print("Destination Port:", packet[TCP].dport)

        elif packet.haslayer(UDP):
            print("Protocol Type: UDP")
            print("Source Port:", packet[UDP].sport)
            print("Destination Port:", packet[UDP].dport)

        elif packet.haslayer(ICMP):
            print("Protocol Type: ICMP")

        # Print payload (if exists)
        if packet.haslayer(TCP):
            payload = bytes(packet[TCP].payload)
            print("Payload:", payload)

# Capture packets
print("Starting Packet Sniffer...")
sniff(prn=packet_callback, count=10)
```

Ln: 20 Col: 36

```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ========== RESTART: Z:\Arvind\Py.py ==========
Starting Packet Sniffer...

--- New Packet Captured ---
Source IP: 192.168.0.107
Destination IP: 192.168.0.104
Protocol: 17
Protocol Type: UDP
Source Port: 57422
Destination Port: 37220

--- New Packet Captured ---
Source IP: 192.168.0.107
Destination IP: 192.168.0.104
Protocol: 17
Protocol Type: UDP
Source Port: 58819
Destination Port: 48995

--- New Packet Captured ---
Source IP: 192.168.0.104
Destination IP: 192.168.0.107
Protocol: 17
Protocol Type: UDP
Source Port: 37220
Destination Port: 57422

--- New Packet Captured ---
Source IP: 192.168.0.104
Destination IP: 192.168.0.107
Protocol: 17
Protocol Type: UDP
Source Port: 48995
Destination Port: 58819

--- New Packet Captured ---
Source IP: 66.117.22.190
Destination IP: 192.168.0.107
Protocol: 6
Protocol Type: TCP
Source Port: 443
Destination Port: 61021
Payload: b''
```

```
IDLE Shell 3.13.7
File  Edit  Shell  Debug  Options  Window  Help
Protocol: 6
Protocol Type: TCP
Source Port: 443
Destination Port: 61021
Payload: b''

--- New Packet Captured ---
Source IP: 192.168.0.107
Destination IP: 66.117.22.190
Protocol: 6
Protocol Type: TCP
Source Port: 61021
Destination Port: 443
Payload: b''

--- New Packet Captured ---
Source IP: 192.168.0.107
Destination IP: 192.168.0.104
Protocol: 17
Protocol Type: UDP
Source Port: 57422
Destination Port: 37220

--- New Packet Captured ---
Source IP: 192.168.0.107
Destination IP: 192.168.0.104
Protocol: 17
Protocol Type: UDP
Source Port: 58819
Destination Port: 48995

--- New Packet Captured ---
Source IP: 192.168.0.104
Destination IP: 192.168.0.107
Protocol: 17
Protocol Type: UDP
Source Port: 37220
Destination Port: 57422

--- New Packet Captured ---
Source IP: 192.168.0.104
Destination IP: 192.168.0.107
Protocol: 17
Protocol Type: UDP
Source Port: 48995
Destination Port: 58819
>>>
```

5. Conclusion

The Basic Network Sniffer project successfully demonstrates packet capturing and analysis using Python. It enhances understanding of networking concepts such as IP addressing, protocol identification, port communication, and payload inspection. This project forms a strong foundation in network security and monitoring.